

If you use Google Gemini for agents, enter GOOGLE_API_KEY; otherwise press Enter.
 GOOGLE_API_KEY (Gemini, optional):
 If you will use CrewAI planning=True (default planner), you may need an OpenAI key
 OPENAI_API_KEY (for built-in planner, optional):

Cell 2 – helpers

```
def extract_yaml_from_text(text: str) -> str | None:
    """
    Tries several strategies to extract a YAML block from an LLM output.
    Returns the YAML text or None.
    """
    if not text:
        return None

    # 1) triple-backtick yaml block
    m = re.search(r"```yaml\s*(.+?)\s*```", text, flags=re.S | re.I)
    if m:
        return m.group(1).strip()

    # 2) triple-backtick block without "yaml"
    m = re.search(r"```(.+?)```", text, flags=re.S)
    if m:
        body = m.group(1).strip()
        # Heuristic: starts with 'agents:' or 'tasks:'
        if body.lstrip().startswith(("agents:", "tasks:")):
            return body

    # 3) raw YAML starting at first 'agents:' token
    idx = text.find("agents:")
    if idx != -1:
        cand = text[idx:]
        # attempt parse progressively
        return cand
    return None

def parse_plan_yaml(yaml_text: str) -> dict:
    """Parse YAML and perform basic validation."""
    try:
        plan = yaml.safe_load(yaml_text)
    except Exception as e:
        raise ValueError(f"Invalid YAML: {e}")
    if not isinstance(plan, dict) or "agents" not in plan or "tasks" not in plan:
        raise ValueError("Plan YAML must be a dict with top-level keys 'agents'")
    # Basic shape checks
    if not isinstance(plan["agents"], list) or not isinstance(plan["tasks"], list):
        raise ValueError("'agents' and 'tasks' must be lists.")
    return plan
```

Cell 3 – create LLM wrapper(s) and Planner agent

Use the LLM you prefer. Example: Gemini via crewai.LLM (if you provided GOOGLE_API_KEY)
 default_model = "gemini/gemini-2.5-flash" if GOOGLE_API_KEY else "openai/gpt-4o-llm_for_agents"
 llm_for_agents = LLM(model=default_model, api_key=GOOGLE_API_KEY or OPENAI_API_KEY)

```
planner_agent = Agent(
    role="Planner",
    goal=(
```

```

        "Given a user request, produce a STRICT YAML plan describing:\n"
        " - agents: list of agent specs (name, role, goal, optional model)\n"
        " - tasks: list of tasks (id, agent (name), description, expected_output)\n"
        "Constraints:\n"
        " - Output only valid YAML (no extra chatty text). If you must include
        " - Keep tasks small and atomic and refer to agents by 'name'."
    ),
    backstory="Experienced project manager. Output concise valid YAML plan only.
    llm=llm_for_agents,
    verbose=True
)

planner_task = Task(
    description="Create a YAML plan for: {user_request}\n\nProduce only the YAML
    expected_output="A YAML plan containing 'agents' and 'tasks'.",
    agent=planner_agent
)

planner_crew = Crew(
    agents=[planner_agent],
    tasks=[planner_task],
    process=Process.sequential,
    verbose=True
)

# Run planner (example request)
user_request = "Build a 3-step scraper: (1) collect top 5 news articles for a topic
planner_result = planner_crew.kickoff({"user_request": user_request})
print("Planner result (raw):")
pprint(planner_result)

```



Crew Execution Started

Crew Execution Started

Name: crew

ID: 832930d2-063f-4066-86ce-577eb0398299

Tool Args:

Agent Started

Agent: **Planner**

Task: Create a YAML plan for: Build a 3-step scraper: (1) collect top 5 news and extract headlines + summary; (3) produce one-paragraph digest.

Produce only the YAML.



Crew: crew

Task: 9bcfc1f8-6c79-407e-8ca6-e6e639a112b8

Status: Executing Task...

Agent Final Answer

Agent: **Planner**

Final Answer:

```
```yaml
agents:
 - name: Collector
 role: Web scraping engineer
 goal: Collect a list of relevant news article URLs for a given topic.
 - name: Extractor
 role: Information extraction specialist
 goal: Parse web pages to extract specific data like headlines and summaries.
 - name: DigestGenerator
 role: Content summarization expert
 goal: Condense multiple pieces of information into a concise, coherent digest.
tasks:
 - id: collect_articles
 agent: Collector
 description: Scrape the top 5 news article URLs for a specified topic (e.g. from a reputable news source).
 expected_output: "A list of 5 URLs to news articles on the specified topic."
 - id: extract_content
 agent: Extractor
 description: For each URL collected, visit the page and extract the main headlines and two paragraphs as a summary.
 dependencies: [collect_articles]
 expected_output: "A JSON array of objects, where each object contains 'url' for one article."
 - id: generate_digest
 agent: DigestGenerator
 description: Review the extracted headlines and summaries from all 5 articles and produce a single, cohesive, one-paragraph digest that captures the essence of the news.
 dependencies: [extract_content]
 expected_output: "A single paragraph (string) summarizing the key points from the news."
```
```

Task Completion

Task Completed

Name: 9bcfc1f8-6c79-407e-8ca6-e6e639a112b8
Agent: Planner
Tool Args:

Crew Completion

Crew Execution Completed

Name: crew
ID: 832930d2-063f-4066-86ce-577eb0398299
Tool Args:
Final Output: ``yaml
agents:
 - name: Collector
 role: Web scraping engineer
 goal: Collect a list of relevant news article URLs for a given topic.
 - name: Extractor
 role: Information extraction specialist
 goal: Parse web pages to extract specific data like headlines and summaries
 - name: DigestGenerator
 role: Content summarization expert
 goal: Condense multiple pieces of information into a concise, coherent digest
tasks:
 - id: collect_articles
 agent: Collector
 description: Scrape the top 5 news article URLs for a specified topic (e.g. from a reputable news source).
 expected_output: "A list of 5 URLs to news articles on the specified topic"
 - id: extract_content
 agent: Extractor
 description: For each URL collected, visit the page and extract the main headlines and two paragraphs as a summary.
 dependencies: [collect_articles]
 expected_output: "A JSON array of objects, where each object contains 'url' for one article."
 - id: generate_digest
 agent: DigestGenerator
 description: Review the extracted headlines and summaries from all 5 articles, create a single, cohesive, one-paragraph digest that captures the essence of the news.
 dependencies: [extract_content]
 expected_output: "A single paragraph (string) summarizing the key points of the news."
 ...

Execution Traces

 Detailed execution traces are available!

View insights including:

- Agent decision-making process
- Task execution flow and timing
- Tool usage details

Would you like to view your execution traces? [y/N] (20s timeout): y

```

# Cell 4 – extract the YAML produced (robustly) and parse it
# The exact shape of planner_result depends on CrewAI version / verbose mode.
planner_text = None
# attempt common keys
if isinstance(planner_result, dict):
    # try a few likely places (examples from different versions)
    for k in ("output", "result", "final_answer", "final_response", "answer"):
        if k in planner_result and isinstance(planner_result[k], str):
            planner_text = planner_result[k]
            break
    # fallback: stringify the whole result
    if planner_text is None:
        planner_text = str(planner_result)

# now extract YAML
yaml_block = extract_yaml_from_text(planner_text)
if not yaml_block:
    print("No YAML found in planner output. Here's the raw text for debugging:")
    print(planner_text)
else:
    try:
        plan = parse_plan_yaml(yaml_block)
        print("Parsed plan:")
        pprint(plan)
    except Exception as e:
        print("Failed to parse plan:", e)
        print("YAML candidate:")
        print(yaml_block)
        raise

```

```

No YAML found in planner output. Here's the raw text for debugging:
None

```

```

# Cell 5 – fallback/demo plan (use this if planner fails to produce valid YAML)
FALLBACK_PLAN = """
agents:
  - name: researcher
    role: Researcher
    goal: "Collect top-5 news article URLs and short extracts for the given topic"
    model: "gemini/gemini-2.5-flash"
  - name: extractor
    role: Extractor
    goal: "From article URLs, extract headline and 2-sentence summary from each"
    model: "gemini/gemini-2.5-flash"
  - name: summarizer
    role: Summarizer
    goal: "Produce a one-paragraph digest based on extracted headlines and summaries"
    model: "gemini/gemini-2.5-flash"

tasks:
  - id: t1
    agent: researcher
    description: "Search web for top 5 articles on {topic} and return list of URLs"
    expected_output: "List[ {url: str, title: str} ]"

```

```

- id: t2
  agent: extractor
  description: "Fetch each URL from t1 and extract headline and 2-sentence sum
  expected_output: "List[{'url', headline, summary}]"
- id: t3
  agent: summarizer
  description: "Create a one-paragraph digest from t2 outputs."
  expected_output: "One paragraph"
"""

# Use fallback if plan variable doesn't exist
if 'plan' not in globals():
    plan = parse_plan_yaml(FALLBACK_PLAN)
    print("Using FALLBACK_PLAN (parsed):")
    pprint(plan)

```

```

Using FALLBACK_PLAN (parsed):
{'agents': [{'goal': 'Collect top-5 news article URLs and short extracts for '
                    'the given topic.',
              'model': 'gemini/gemini-2.5-flash',
              'name': 'researcher',
              'role': 'Researcher'},
            {'goal': 'From article URLs, extract headline and 2-sentence '
                    'summary from each page.',
              'model': 'gemini/gemini-2.5-flash',
              'name': 'extractor',
              'role': 'Extractor'},
            {'goal': 'Produce a one-paragraph digest based on extracted '
                    'headlines and summaries.',
              'model': 'gemini/gemini-2.5-flash',
              'name': 'summarizer',
              'role': 'Summarizer'}],
 'tasks': [{'agent': 'researcher',
            'description': 'Search web for top 5 articles on {topic} and '
                          'return list of URLs + one-sentence title each.',
            'expected_output': 'List[ {url: str, title: str} ]',
            'id': 't1'},
            {'agent': 'extractor',
            'description': 'Fetch each URL from t1 and extract headline and '
                          '2-sentence summary.',
            'expected_output': 'List[{'url', headline, summary}]',
            'id': 't2'},
            {'agent': 'summarizer',
            'description': 'Create a one-paragraph digest from t2 outputs.',
            'expected_output': 'One paragraph',
            'id': 't3'}]}

```

```

# Cell 6 – instantiate Agents + Tasks for executor crew
# map plan agents to actual crewai.Agent objects
agent_map = {}
default_llm = llm_for_agents # reuse planner LLM (customize per agent if model f

for a in plan["agents"]:
    name = a.get("name")
    model_key = a.get("model")
    model_llm = default_llm
    if model_key:
        # create a per-agent LLM if a different model is requested (use same key)
        model_llm = LLM(model=a.get("model"), api_key=(GOOGLE_API_KEY or OPENAI_A

```



```

agent_obj = Agent(
    role=a.get("role", name),
    goal=a.get("goal", ""),
    backstory=a.get("backstory", ""),
    llm=model_llm,
    verbose=True
)
agent_map[name] = agent_obj

# create Task objects in the order listed (simple sequential runner)
task_objs = []
for t in plan["tasks"]:
    agent_name = t["agent"]
    if agent_name not in agent_map:
        raise ValueError(f"Agent named '{agent_name}' not defined in plan agents.")
    task_objs.append(Task(
        description=t["description"].format(topic="AI regulation"),
        expected_output=t.get("expected_output", ""),
        agent=agent_map[agent_name]
    ))

# Build the executor crew
executor_crew = Crew(
    agents=list(agent_map.values()),
    tasks=task_objs,
    process=Process.sequential,
    verbose=True
)

# Human-in-the-loop gating: ask user for permission before executing (recommended
do_execute = True # change to False to skip
if do_execute:
    print("Running executor crew...")
    exec_result = executor_crew.kickoff({"topic": "AI ethics"}) # provide context
    print("Executor result (raw):")
    pprint(exec_result)
else:
    print("Execution skipped. Inspect 'plan' and agent/task definitions before ru

```



Running executor crew...

Crew Execution Started

Crew Execution Started
Name: crew
ID: abe1869f-4f22-47b5-ab3e-1ec9ec280970
Tool Args:

Agent Started

Agent: **Researcher**

Task: Search web for top 5 articles on AI regulation and return list of URLs +

 **Crew: crew**
 **Task: c566b599-cfe8-44ac-b66a-8e61f116f223**
Status: Executing Task...

Agent Final Answer

Agent: **Researcher**

Final Answer:
[
 {
 "url": "https://www.brookings.edu/articles/ai-regulation-in-the-us-an-
 "title": "AI regulation in the U.S.: An update"
 },
 {
 "url": "https://www.cfr.org/middle-east-and-north-africa/ai-regulation
 "title": "AI Regulation Around the World"
 },
 {
 "url": "https://hbr.org/2023/12/why-ai-regulation-is-so-hard-and-what-
 "title": "Why AI Regulation Is So Hard – And What to Do About It"
 },
 {
 "url":
 "https://www.euractiv.com/section/artificial-intelligence/news/eu-ai-act-final
 knesses/",
 "title": "EU AI Act final text reveals strengths and weaknesses"
 },
 {
 "url":
 "https://www.whitehouse.gov/briefing-room/statements-releases/2023/10/30/presi
 utive-order-on-safe-secure-and-trustworthy-artificial-intelligence/",
 "title": "President Biden Issues Landmark Executive Order on Safe, Sec
 Intelligence"
 }
]

Task Completion

Task Completed
Name: c566b599-cfe8-44ac-b66a-8e61f116f223
Agent: **Researcher**
Tool Args:



Agent Started

Agent: **Extractor**Task: **Fetch each URL from t1 and extract headline and 2-sentence summary.****Crew: crew****Task: c566b599-cfe8-44ac-b66a-8e61f116f223**Assigned to: **Researcher**Status: **Completed****Task: 2164e380-c9e3-4058-b201-a3ff3e0736e2**Status: **Executing Task...**

Agent Final Answer

Agent: **Extractor**

Final Answer:

[